Laboratorium 1: parser strony www

Rola: PARSER

Autor parsera: Michał Langner

Autor serwisu: Jakub Nawrocki

Opis serwisu

Opis przygotowany na podstawie dokumentacji autora serwisu (Jakub Nawrocki).

Struktura serwisu

Bajtomarket to fikcyjny katalog 420 produktów. Serwis jest statyczny, ale cała treść listy i kart produktów powstaje w przeglądarce: w źródle HTML nie ma ani jednego rekordu, a dane pochodzą z plików JSON pobieranych przez `fetch`.

- **Podstrony katalogu:** 6 podstron (`index.html`, `strona-2.html` … `strona-6.html`) po 72 pozycje, razem 432 pozycje (420 unikalnych rekordów).
- **Dwie struktury tej samej listy:** tabela `table.tabela-ofert` oraz karty `article.produkt` na jednej podstronie.
- **Doładowywanie:** przycisk „Wczytaj więcej", porcje po 12 rekordów, 6 porcji na podstronę (36 plików JSON z porcjami). Przed pokazaniem każdej porcji serwis losuje opóźnienie 300–2000 ms.
- **Strony produktów:** własna strona `produkt/<id>.html` dla każdego z 420 rekordów, także bez rekordu w źródle; pełne dane w `dane/produkt/<id>.json`.
- **Atrybuty:** 26, z czego 13 na liście i 13 dodatkowych na stronie produktu.

Sposób generowania

Dane i cały serwis tworzy jednorazowo generator `generuj_serwis.py`. Dane są w całości fikcyjne i generowane deterministycznie z ziarna 2026, więc wynik da się odtworzyć co do rekordu. Generator uruchomiony z innym ziarnem buduje serwis od nowa.

Zrealizowane utrudnienia

Poziom 4.0 (wymagane 3, zrealizowano 3):

1. *Ceny w różnych formatach i walutach.* 357 rekordów w złotych (`1 299,00 zł` z twardą spacją U+00A0), 38 w euro (`246,99 €`), 25 w dolarach (`$134.99`). Przecinek bywa separatorem dziesiętnym albo tysięcy, więc naiwne zastąpienie przecinka kropką zawyża cenę dolarową stukrotnie. Ten sam błąd kontrola jakości danych ma wychwycić jako wartość odstającą.
2. *Braki wartości w dwóch polach.* `producent` brakuje w 71 rekordach (16,9%), `ocena` w 84 (20,0%); brak to `–` z klasą `brak-wartosci`. Pusta cena poprzednia oznacza brak promocji, nie brak danych, więc parser musi rozróżnić te przypadki.
3. *Duplikaty.* 12 zduplikowanych rekordów rozrzuconych po podstronach, po dwa na podstronę, zawsze na innej niż oryginał, co wymaga deduplikacji po całej liście.

Poziom 5.0 (wymagane 2, zrealizowano 2):

1. *Atrybuty widoczne dopiero po interakcji.* Zakładki *Specyfikacja* i *Dostawa i zwroty* wypełniają się przy pierwszym kliknięciu; wcześniej oba panele są w DOM puste, więc 12 atrybutów jest nieosiągalnych bez kliknięcia.
2. *Filtrowanie i sortowanie bez przeładowania.* Wybór kategorii lub porządku przerysowuje tabelę i karty z rekordów już wczytanych, bez nowego żądania.

Mapa atrybutów w skrócie

Lista (13 atrybutów, te same w tabeli i kartach): `id` (`data-id`), `nazwa` i `url` (`a.odnosnik-produktu`), `cena` i `waluta` (`.cena`, `data-waluta`), `cena_poprzednia`, `kategoria`, `producent`, `dostepnosc`, `gwarancja`, `ocena`, `liczba_opinii` i `data_dodania` (format `DD.MM.RRRR`), każdy w klasie o tej samej nazwie.

Strona produktu (13 dodatkowych atrybutów): opis jest widoczny od razu (`#panel-opis`). Pozostałe 12 pojawia się po kliknięciu zakładki: *Specyfikacja* (`ean`, `kod_producenta`, `kolor`, `material`, `waga`, `wymiary`, `kraj_pochodzenia`) oraz *Dostawa i zwroty* (`sprzedawca`, `stan_magazynowy`, `czas_dostawy`, `koszt_dostawy`, `zwrot`).

Znaczniki stanu: `body[data-wczytano]`, `body[data-widocznych]`, `body[data-porcja]` i `body[data-porcji-lacznie]`, `body[data-wczytywanie="tak"]`, `body[data-stan-renderu="gotowy"]` oraz `#panel-specyfikacja[data-wypelnione="tak"]`.

1. Użyta technologia i uzasadnienie wyboru

Selenium (Chrome). Surowy HTML serwisu nie zawiera żadnego rekordu: pobrany bez przeglądarki (`urllib`) ma 0 wystąpień `tr.wiersz-produktu`, 0 `article.produkt` i 0 `data-id` na wszystkich 6 podstronach listy (po 3952 B każda, 2 znaczniki `<script>`), a w DOM po wykonaniu skryptów jest 432 pozycje (`output/okno_porownanie.json`). Dane trafiają do DOM przez JavaScript, więc potrzebna jest przeglądarka. Dodatkowo 12 z 13 atrybutów szczegółów produktu (zakładki *Specyfikacja* i *Dostawa i zwroty*) nie istnieje w DOM przed kliknięciem zakładki: 20/20 sprawdzonych produktów miało 0 wypełnionych pól przed kliknięciem i odpowiednio 7 i 5 po (`docs/NOTATKI-DO-SPRAWOZDANIA.md`, faza 5).

pandas. Buduje ramkę o stałej kolejności 30 kolumn (`parsing/rekord.py: KOLUMNY`) i zapisuje CSV (UTF-8) oraz JSON (`force_ascii=False`) jednym wywołaniem (`eksport.py`). Kolumny całkowite mają typ `Int64`, więc braki nie zamieniają liczb całkowitych na zmiennoprzecinkowe (np. 25 pozostaje 25).

Page Object Model (ścisły). Selektory, klikanie, nawigacja i oczekiwania są wyłącznie w `bajtoparser/pages/`; `main.py`, `cli.py` i `pipeline.py` nie importują `selenium`. Page Objecty zwracają surowe napisy, a czyszczenie (ceny, daty, braki) jest w `parsing/` bez Selenium. Dzięki temu 240 testów pytest działa bez przeglądarki (2,61 s), w tym 48 testów funkcji czyszczących. Wybór uzasadnia też weryfikacja: niezależny odczyt surowych napisów wobec CSV dał 0 niezgodnych w 7980 porównaniach (`analiza/wyniki/kontrola_jakosci.md`).

Pozostałe elementy stosu: argparse (CLI), pytest, ruff. `matplotlib` służy wyłącznie do wykresów w `analiza/`. Wybrano Chrome, bo Selenium Manager sam dobiera i pobiera sterownik; nie porównywano go z innymi przeglądarkami.

2. Opis parsera: rozszerzenia i selektory

2.1 Wybrane rozszerzenia

Poziom	Rozszerzenie	Realizacja
4.0	Parametryzacja z CLI	`--adres`, `--strony`, `--limit`, `--headless`, `--porownanie`, `--wyjscie`, `--katalog-wyjscia`, `--pauza`, `--timeout` (`cli.zbuduj_parser`)
4.0	Eksport CSV + JSON	oba pliki jednocześnie (`eksport.eksportuj`)
4.0	Logowanie z podsumowaniem	`logs/bajtoparser_<znacznik>.log`, linia `PODSUMOWANIE:` (strony, rekordy, błędy, retry, czas)
4.0	Testy funkcji czyszczących	`tests/test_ceny.py`, `test_daty_braki.py`, `test_pola.py`, `test_duplikaty.py`, `test_rekord.py` (48 testów)
5.0	Headless z pomiarem czasu	`--headless`, linia `PRZEBIEG:` w obu trybach (sekcja 6)
5.0	Diagnostyka błędów	po wyczerpaniu prób `.png` + `.html` do `diagnostyka/`, potem wyjątek dalej

Rozszerzenie z listy 5.0 „wykrycie i wykorzystanie źródła danych (plik JSON)" zostało świadomie pominięte i nie jest zaimplementowane. Wymagane są co najmniej 2 rozszerzenia z listy 5.0; wybrane i zrealizowane są

dokładnie te dwa (headless z pomiarem czasu oraz diagnostyka). Rozszerzenia 429/Retry-After, checkpoint i porównanie pamięci są poza zakresem (serwis nie ma limitu żądań).

Diagnostyka zapisuje `.png` i `.html` po wyczerpaniu prób danej operacji (a nie po każdej nieudanej próbie), żeby ponawianie nie mnożyło plików; zapis następuje tuż przed rzuceniem wyjątku dalej.

2.2 Przebieg działania

Parser otwiera podstronę listy, doładowuje porcje po 12 rekordów do wyczerpania (72 pozycje na podstronę, 6 podstron), zbiera rekordy z tabeli i z kart w jednym przejściu, usuwa duplikaty po `id` **przed** wejściem w szczegóły (432 → 420) i dla każdego unikalnego produktu otwiera zakładki *Specyfikacja* i *Dostawa i zwroty*. Wynik: 420 rekordów, 26 atrybutów serwisu, 30 kolumn (cena rozbita na `cena` + `waluta`, wymiary na trzy kolumny, koszt dostawy na `koszt_dostawy` + `waluta_dostawy`, plus `zrodlo`: tabela 178 / karta 242).

2.3 Selektory i uzasadnienie

Pełna mapa: `docs/SELEKTORY.md`. Selektory są stałymi na górze klas stron (`pages/catalog_page.py`, `pages/product_page.py`).

Element	Selektor	Dlaczego ten
wiersz tabeli / karta	`tr.wiersz-produktu`, `article.produkt`	Klasy semantyczne. Pozycja kolumny w tabeli nie jest kontraktem, więc `td:nth-child` i XPath odrzucone. Tabela i karty to rozłączne zbiory (30 + 42 = 72 na podstronę, 0 wspólnych `data-id`), więc jedna struktura nie wystarczy.
identyfikator	atrybut `data-id`	Atrybut danych, używany też przez skrypt serwisu.
waluta	atrybut `data-waluta`	Pewniejsza niż symbol w tekście ceny.
pola listy	`.cena`, `.cena-poprzednia`, `.kategoria`, `.producent`, `.dostepnosc`, `.gwarancja`, `.ocena`, `.liczba-opinii`, `.data-dodania`	Te same klasy w obu strukturach. Prefiks `.etykieta` w kartach jest wycinany w warstwie stron.
nazwa i URL	`a.odnosnik-produktu` (`href`)	Klasa semantyczna.
doładowywanie	`#wczytaj-wiecej`	Id stabilne i unikalne; tekst przycisku zmienny, więc nie na nim.
stan doładowywania	`body[data-wczytano]`, `[data-porcja]`, `[data-porcji-lacznie]`	Oczekiwanie na **wzrost** `data-wczytano`; koniec, gdy porcja = porcji łącznie. Flaga `data-wczytywanie` migocze 0,3-2 s, więc nie jest używana.
paginacja	`a[href*="strona-"]`	Liczba podstron ustalana z linków (6), nie stała.
gotowość produktu	`body[data-stan-renderu="gotowy"]`	Znacznik stanu aplikacji.
zakładki	`button.zakladka[data-panel="specyfikacja|dostawa"]`	`data-panel` jest kluczem niezależnym od napisu na przycisku.
zakładka otwarta	`#panel-specyfikacja[data-wypelnione="tak"]`, `#panel-dostawa[data-wypelnione="tak"]`	Marker potwierdzony w DOM (identyczny dla obu paneli).
pola szczegółów	`dd.<pole>` w panelach (`.ean`, `.kod-producenta`, `.waga`, `.wymiary`, `.sprzedawca`, `.koszt-dostawy` …)	Nazwa klasy = nazwa pola. Odczyt `textContent`, bo ukryty panel zwraca `''` dla `.text`.

Decyzje techniczne:

1. Rekordy listy są zbierane jednym `execute_script` na podstronę zamiast `find_element` dla każdego pola: wariant klasyczny to ok. 900 wywołań WebDrivera na podstronę (72 rekordy × 13 pól) i ryzyko

`StaleElementReferenceException`; skrypt daje 1 wywołanie i spójną migawkę (`docs/NOTATKI-DO-SPRAWOZDANIA.md`, faza 4). Selektory przekazuje Python, więc zasada „selektory w klasie strony" zostaje zachowana.

2. Retry (1, 2, 4 s; 4 próby; tylko `TimeoutException`, `StaleElementReferenceException`, `ElementClickInterceptedException`) stoi na metodach najwyższego poziomu i nie jest zagnieżdżany, żeby próby się nie mnożyły, a diagnostyka zapisywała się raz. W obu pełnych przebiegach retry nie zadziałał ani razu (`retry=0`, `błędów=0`); zachowanie 1/2/4 s potwierdzają testy (`tests/test_retry.py`, `tests/test_timing.py`) i jedna kontrolowana próba na nieistniejącym selektorze (4 próby, opóźnienia 1/2/4 s, jedna para plików diagnostycznych).
3. Gdy serwis nie odpowiada, program kończy się kodem wyjścia 2 z komunikatem „Serwis nie odpowiada pod adresem http://localhost:8000/ ...", bez uruchamiania Chrome'a (sprawdzenie jednym żądaniem HTTP przed otwarciem przeglądarki). Zweryfikowane ręcznie przy zatrzymanym serwerze (`python main.py --strony 1 --limit 1 --headless`) oraz testem jednostkowym.

3. Ocena odporności selektorów

Zastrzeżenie: to ocena na podstawie postaci selektorów i ich oparcia w kontrakcie serwisu, nie wynik eksperymentu. Parser nie był uruchamiany na zmodyfikowanej wersji serwisu.

Przyjęto dwa rodzaje zmian: (A) zmiana wyglądu bez zmiany struktury i nazw (inny CSS, kolory, kolejność kolumn, układ kart), (B) zmiana struktury lub nazw (przemianowanie klas, zmiana tagu, zmiana schematu adresów, zmiana atrybutów `data-*`).

Selektor	Zmiana wyglądu (A)	Zmiana struktury/nazw (B)	Uzasadnienie
`data-id`, `data-waluta`, `data-panel`	przetrwa	przetrwa, jeśli kontrakt z JS serwisu pozostaje; zawiedzie przy przemianowaniu atrybutu	Atrybuty danych nie służą stylowaniu.
`body[data-wczytano]`, `data-porcja`, `data-porcji-lacznie`, `data-stan-renderu`, `data-wypelnione`	przetrwa	zawiedzie przy zmianie nazwy lub semantyki znacznika	Znaczniki stanu aplikacji, niezależne od wyglądu; parser jest na nich oparty.
`#wczytaj-wiecej`	przetrwa	zawiedzie przy zmianie id	Id jest unikalne i niezwiązane ze stylem; zmiana tekstu przycisku nie szkodzi.
`tr.wiersz-produktu`, `article.produkt`	przetrwa	zawiedzie przy zmianie klasy **lub tagu** (np. `div` zamiast `article`)	Selektor łączy tag i klasę, więc zależy od dwóch rzeczy. Przetrwa zmianę kolejności kolumn, bo pola są po klasach, nie po pozycji.
`.cena`, `.kategoria`, `.producent`, `.ocena` …	przetrwa	zawiedzie przy zmianie nazw klas	Klasy semantyczne nazwane po polu. Ryzyko: klasa mogłaby zostać użyta w innym celu stylistycznym; wtedy selektor złapie dodatkowe elementy.
`a.odnosnik-produktu`	przetrwa	zawiedzie przy zmianie klasy	Jak wyżej. `href` w postaci `/produkt/<id>.html` jest zależny od schematu adresów.
`.etykieta` (wycinana z kart)	przetrwa	zawiedzie przy zmianie klasy; wtedy prefiks „Cena:" zostałby w wartości	Błąd byłby widoczny, bo napis przestałby się parsować (rośnie liczba „nierozpoznanych").
`a[href*="strona-"]`	przetrwa	zawiedzie przy zmianie schematu adresów (np. `?page=2`)	Zależy od nazwy pliku w adresie, nie od wyglądu.

Selektor	Zmiana wyglądu (A)	Zmiana struktury/nazw (B)	Uzasadnienie
`dd.<pole>` w panelach	przetrwa	zawiedzie przy zmianie tagu `dd` lub nazw klas	Tag `dd` i klasa polu; kolejność pól nie ma znaczenia.
`button.zakladka[...]`	przetrwa	zawiedzie przy zmianie klasy `zakladka` lub tagu	`data-panel` stabilne, ale w selektorze są też tag i klasa.
`body` (tag)	przetrwa	przetrwa	Element obecny w każdym dokumencie.

Co by nie przetrwało na pewno (zmiana wyglądu, która wymusza zmianę struktury): przeniesienie etykiet z `<span class="etykieta">` do pseudoelementu CSS lub zmiana tabeli na listę `div` (zmiana tagu `tr`). **Pozycyjne selektory** (`td:nth-child`, XPath po indeksie) zostały świadomie odrzucone, bo to właśnie one zawodziłyby przy najprostszej zmianie układu.

Ogólny wniosek: parser jest odporny na zmiany czysto wizualne (A), a podatny na zmiany nazw klas i atrybutów (B), przy czym znaczniki `data-*` są najstabilniejsze. Wartości, które przestałyby się parsować, nie zostałyby po cichu zamienione na `NaN`: pipeline liczy i loguje „nierozpoznane" wartości (wartość niebędąca brakiem, a po czyszczeniu `None`); w obu pełnych przebiegach było ich 0.

4. Wyniki kontroli jakości danych

Źródło: `analiza/wyniki/kontrola_jakosci.md` (generowany przez `analiza/kontrola_jakosci.py`).

Liczba rekordów i braki (420 rekordów, 30 kolumn):

kolumna	braków (szt.)	braków (%)	uwaga
producent	71	16,9	`–` w surowych
ocena	84	20,0	`–` w surowych
gwarancja_mies	21	5,0	`–` w surowych
waga_kg	25	6,0	`–` w surowych
cena_poprzednia	329	78,3	brak promocji, nie brak danych

Pozostałe 25 kolumn bez braków. Braki w CSV są równe liczbie `–` w surowych danych dla każdej kolumny.

Duplikaty: 432 pozycje na liście → 420 po deduplikacji po `id` (usunięto 12); 0 duplikatów różniło się od oryginału (porównanie w pipeline przed usunięciem, liczba z logu `logs/bajtoparser_20261018_165547.log`). W `okno.csv` 0 powtórzonych `id` i 0 powtórzonych `url`. Ograniczenie: usunięte duplikaty nie są zapisane w żadnym pliku, więc porównania nie da się powtórzyć z samych plików wyjściowych.

Typy kolumn: wszystkie 30 kolumn zgodne z oczekiwaniem; 0 tekstu w kolumnach liczbowych. Kontrole spójności (EAN 13 cyfr, ocena 0-5, liczby ujemne, URL zgodny z `id`, data ISO, waluta ze zbioru, znaki zastępcze): 0 naruszeń.

Wartości odstające (IQR i z-score; waluty osobno):

cecha	n	odstające IQR	odstające |z|>3	max	interpretacja
cena PLN	357	37	7	24999	prawostronny ogon (drogi sprzęt); surowy napis `24 999,00 zł`
cena EUR	38	2	1	786,99	j.w.
cena USD	25	3	1	1769,99	j.w.; `$1,769.99` odczytane poprawnie

cecha	n	odstające IQR	odstające |z|>3	max	interpretacja
waga_kg	395	47	11	38,5	ciężkie towary; `38,5 kg`
szerokosc_cm	420	17	7	116	duże towary (RTV)
wysokosc_cm	420	45	9	191	lampy stojące (36 x 191 x 47)
glebokosc_cm	420	35	24	203	sprzęt sportowy (71 x 1 x 203)
liczba_opinii	420	38	1	12480	jedna wartość 26,1× większa od następnej (479); same cyfry w surowych
stan_magazynowy_szt	420	0	0	250	90 rekordów z 0 szt. to realny stan, nie brak

Wniosek z tabeli: wszystkie odstające mają odpowiednik w surowym tekście (odczyt niezależny dał te same liczby), więc to cechy danych serwisu, nie błędy parsera. Wartość 12480 w `liczba_opinii` (P86250) to anomalia danych źródłowych.

Test naiwnego `replace(",", ".")` na rzeczywistych surowych cenach: poprawny dla 357/357 PLN i 38/38 EUR; dla USD 23/25 poprawnych, 2 rekordy z separatorem tysięcy (`$1,471.99`, `$1,769.99`) dają `ValueError` (`1.471.99`). Zawyżenie 100× daje dopiero wariant „usuń przecinki" zastosowany do PLN/EUR (`139,00 zł` → 13900).

Niezależny odczyt surowych napisów wobec CSV: 0 niezgodnych w 7980 porównaniach (19 pól × 420 rekordów). Liczby zgodne z README serwisu (432, 420, 12; waluty 357/38/25; braki 71 i 84). README nie podaje braków `gwarancja_mies` (21) i `waga_kg` (25).

Zgodność okno vs headless: CSV, JSON i JSONL mają te same SHA-256 (`DataFrame.equals`: True, 420 × 30).

Wniosek o parserze: nie znaleziono błędu w wartościach (0 niezgodnych w 7980 porównaniach), więc jedyną poprawką w `parsing/` jest zmiana nazw kolumn wymiarów (niżej).

Poprawka wynikająca z analizy: nazwy kolumn wymiarów. Serwis nie podpisuje trzech liczb w polu `wymiary` (np. `17 x 9 x 8 cm`), a parser pierwotnie przypisał je kolejno do `dlugosc_cm`, `szerokosc_cm`, `wysokosc_cm`. Rozkłady wymiarów (lampa `36 x 191 x 47`, telefon `7 x 14 x 1`) wskazały, że kolejność jest inna niż założona: druga liczba lampy (191 cm) to wysokość, a nie szerokość. Kod generatora serwisu to potwierdził: liczby w `A x B x C cm` to kolejno szerokość, wysokość i głębokość. Kolumny nazwano więc `szerokosc_cm`, `wysokosc_cm` i `glebokosc_cm` (1., 2. i 3. liczba). Wartości były przepisane poprawnie, zmieniły się tylko nazwy i opisy; nazwa `szerokosc_cm` występuje w obu wersjach, ale dotyczy teraz 1. liczby (dawniej 2.). Tabela odstających wyżej używa nowych nazw.

5. Statystyki i wnioski

Źródło: `analiza/wyniki/statystyki.md` (generowany przez `analiza/statystyki.py`).

Próba. Ceny analizowane osobno według walut, bez przeliczania kursem (brak źródłowego kursu; każdy kurs byłby arbitralnym założeniem). PLN: 357 rekordów (85,0%), EUR 38, USD 25. Pozostałe cechy liczone na wszystkich 420 rekordach.

Cena (statystyki opisowe):

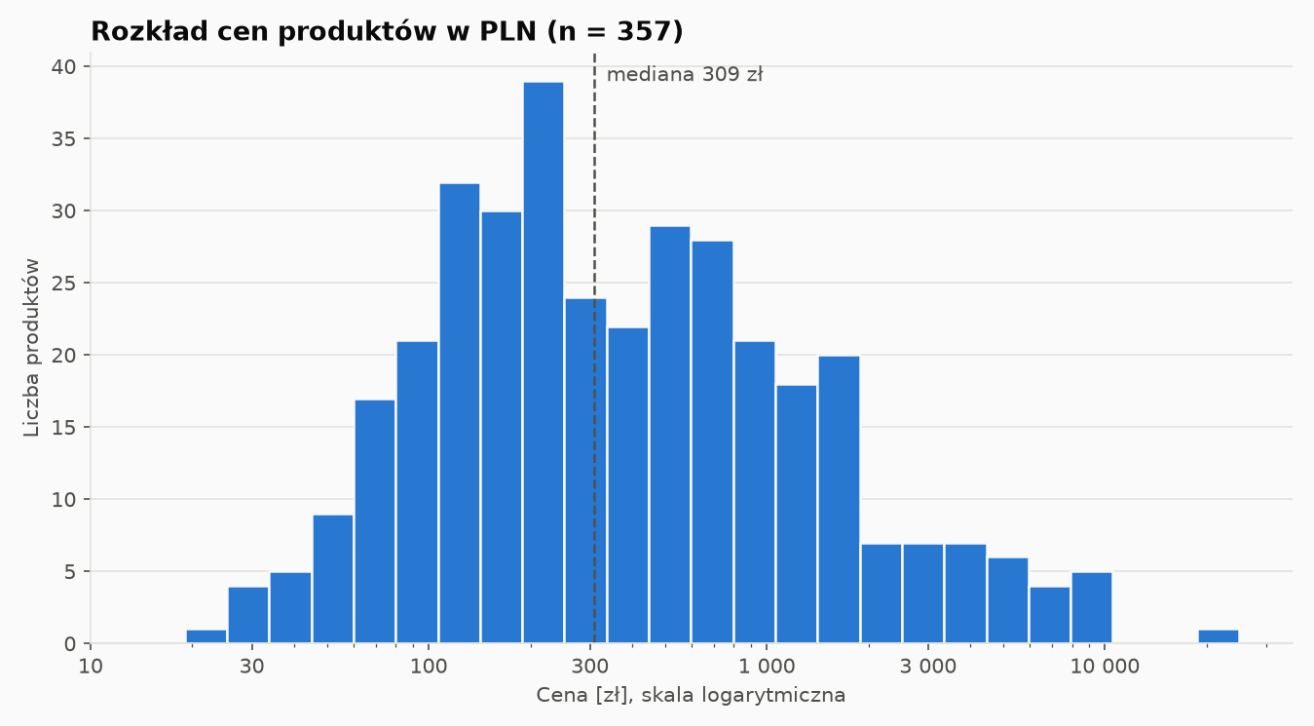
waluta	n	średnia	mediana	Q1	Q3	min	max	odch. std.
PLN	357	931,72	309	149	849	19	24999	1952,76
EUR	38	160,86	60,99	23,49	235,24	6,99	786,99	199,25
USD	25	225,07	83,99	45,99	165,99	6,99	1769,99	430,10

Cena PLN jest silnie prawoskośna (skośność 6,73; średnia 931,72 zł wobec mediany 309 zł), więc do opisu typowej ceny służą mediana i kwartyle.

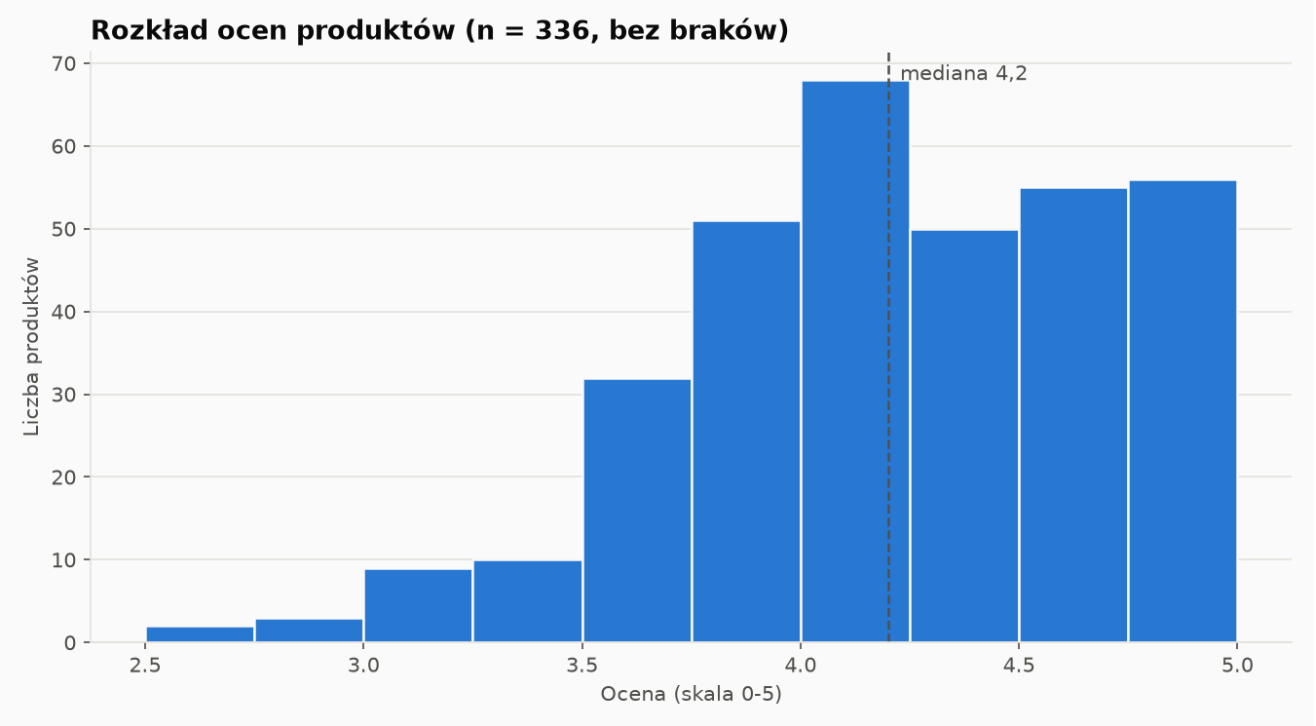
Pozostałe cechy (mediana / Q1 / Q3): ocena (n=336) 4,2 / 3,9 / 4,6; liczba opinii 14 / 3 / 31; gwarancja (n=399) 24 / 24 / 24 mies.; waga (n=395) 1,46 / 0,36 / 3,15 kg; stan magazynowy 52,5 / 2 / 144,5 szt.; czas dostawy 2,5 / 2 / 4 dni; koszt dostawy 9,99 / 0 / 14,99; zwrot 30 / 30 / 30 dni. Pełna tabela (średnia, moda, min, max, odch. std.): sekcja 3 w `statystyki.md`.

Wykresy:

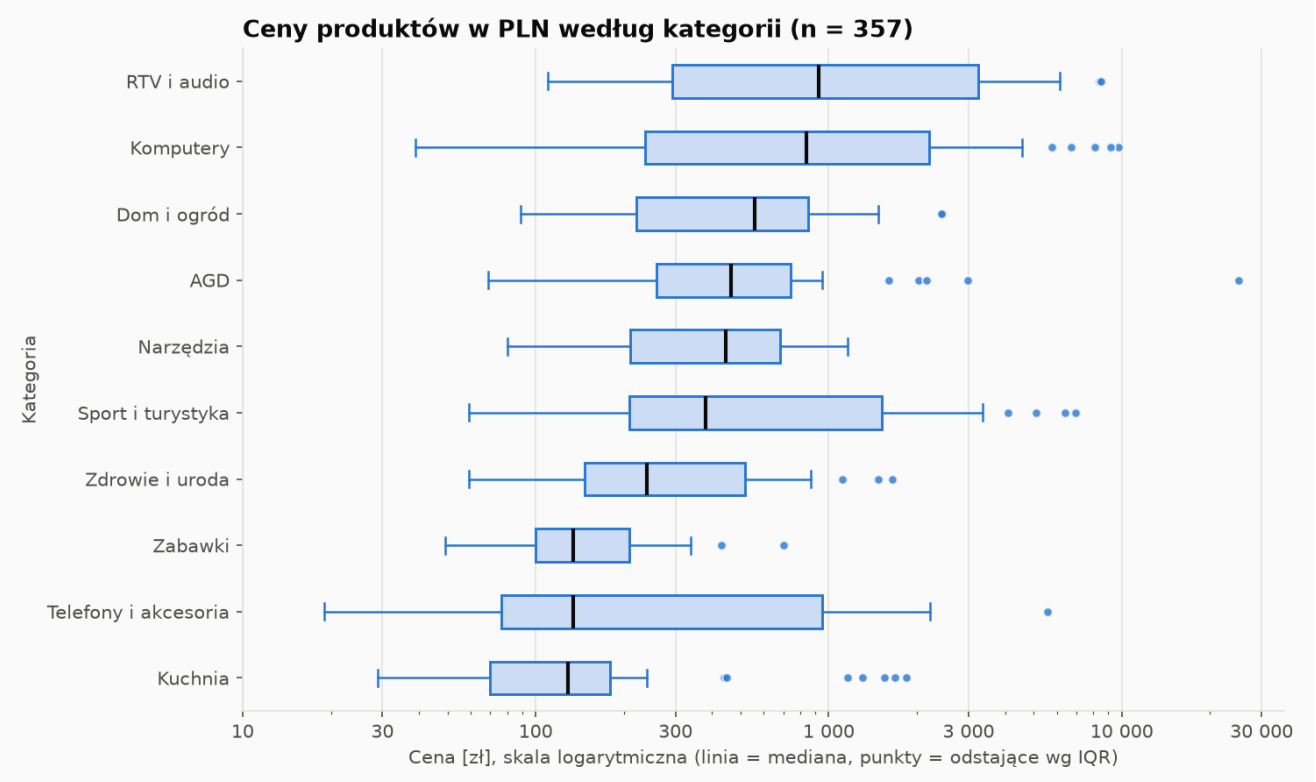
- Histogram ceny PLN (skala logarytmiczna): `analiza/wykresy/hist_cena_pln.png`. Zakres 19-24 999 zł, mediana 309 zł, długi prawy ogon, bez jednego wyraźnego szczytu.
- Histogram oceny: `analiza/wykresy/hist_ocena.png`. Oceny wokół mediany 4,2 (Q1 3,9, Q3 4,6); 84 produkty bez oceny nie są uwzględnione.
- Box-plot ceny PLN według kategorii: `analiza/wykresy/box_cena_pln_kategoria.png`. Najniższa mediana: Kuchnia (129 zł), najwyższa: RTV i audio (924,50 zł), czyli 7,2× więcej. Kategoria wyjaśnia 23,5% wariancji log10(ceny) (eta^2, ANOVA jednoczynnikowa); pudełka kategorii się nakładają, więc sama kategoria nie wyznacza ceny.
- Wykresy rozrzutu korelowanych cech: `analiza/wykresy/korelacje.png`.



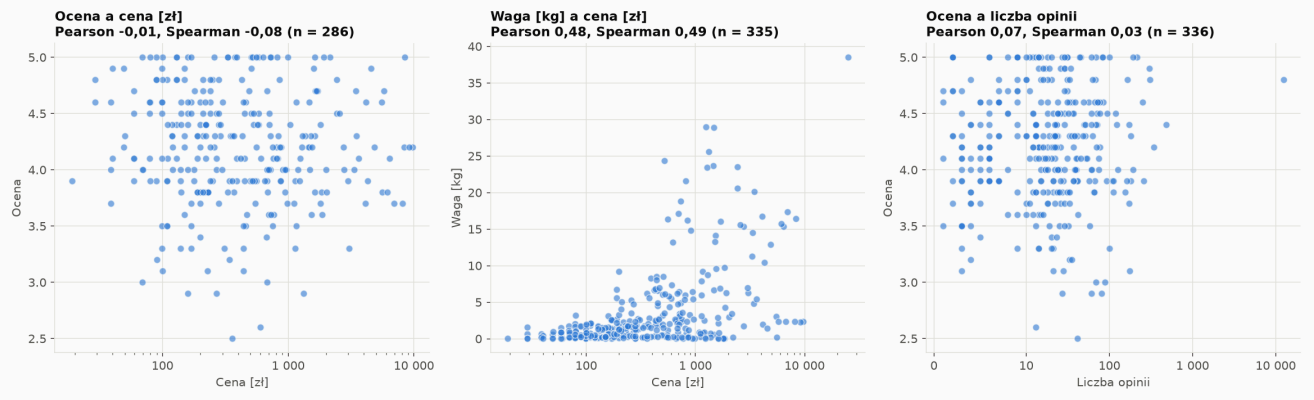
Histogram ceny PLN (skala logarytmiczna)



Histogram oceny



Box-plot ceny PLN według kategorii



Wykresy rozrzutu korelowanych cech

Korelacje (n po usunięciu braków; „istotna" = |r| powyżej przybliżonego progu 1,96/√(n−1)):

para	n	Pearson	Spearman	istotna
cena PLN – ocena	286	-0,01	-0,08	nie
cena PLN – waga_kg	335	0,48	0,49	tak (umiarkowana)
cena PLN – waga_kg (rangi w obrębie kategorii)	335	0,42	0,42	tak (umiarkowana)
liczba_opinii – ocena	336	0,07	0,03	nie
liczba_opinii – ocena (bez max.)	335	0,05	0,02	nie
cena PLN – stan_magazynowy_szt	357	-0,05	-0,03	nie
cena PLN – liczba_opinii	357	-0,02	-0,06	nie

Wnioski.

- Jedyna istotna zależność to cena–waga (Spearman 0,49; po usunięciu efektu kategorii 0,42): cięższe towary są na ogół droższe, ale elektronika bywa lekka i droga, stąd tylko umiarkowana siła. Wspomniane uzasadnienie merytoryczne (AGD, sprzęt sportowy) jest interpretacją, nie wynikiem testu.
- Brak zależności ceny od oceny i liczby opinii od oceny, mimo że na rynku oczekiwano by dodatnich. Dane są fikcyjne i losowo generowane; brak korelacji jest wynikiem do omówienia, nie wadą parsera (niezależny odczyt: 0 niezgodności). Usunięcie rekordu z 12480 opiniami zmienia Pearsona liczby opinii z oceną z 0,068 na 0,048, więc wniosek od niego nie zależy.
- Ograniczenie metodyczne: Spearman jest liczony jako Pearson rang (średnie rangi przy remisach), bez `scipy`, które nie należy do stosu projektu; istotność oceniana przybliżonym progiem, bez p-wartości. Wartości współczynników są poprawne, ale wnioski o istotności należy traktować jako przybliżone.
- Ceny w EUR (38 rekordów) i USD (25) zostały w tabeli, bez wykresów i korelacji, z powodu zbyt małej próby.

6. Porównanie surowego HTML z DOM oraz okno vs headless

6.1 Surowy HTML vs DOM

Źródło: `output/okno_porownanie.json` i `output/headless_porownanie.json` (identyczne liczby w obu trybach), linia `POROWNANIE:` w logu `logs/bajtoparser_20261018_165547.log`.

Miara	Surowy HTML (urllib)	DOM (po skryptach)
Podstrony listy	6 pobranych, po 3952 B	6 przetworzonych
`wiersz-produktu` (tabela)	0	180 pozycji
`produkt` (karty)	0	252 pozycje

Miara	Surowy HTML (urllib)	DOM (po skryptach)
`data-id`	0	brak osobnego pomiaru w DOM
Pozycje na liście razem	0	432
Po deduplikacji	0	420 (usunięto 12)
Rekordy w CSV	0	420
Strony produktów z nazwą i id w treści (z 10)	0	10 z 10 (13/13 pól, faza 5)
`<script>` na liście / na produkcie	2 / 2	n/d

Uwaga do odczytu: w DOM „tabela" i „karty" to liczba pozycji z pola `zrodlo` (przed deduplikacją), nie licznik elementów DOM. W surowym HTML produktu identyfikator występuje tylko w `<meta name="produkt">` i w `<noscript>` (`id_w_html=false`, `id_gdziekolwiek=true` w 10/10 próbkach), nazwy nie ma w żadnym.

Wniosek: w surowym HTML jest 0 rekordów na wszystkich 6 podstronach, a w DOM po skryptach 432 pozycje (180 + 252), czyli 420 unikalnych. Serwis jest statyczną powłoką, a całość danych dostarcza JavaScript, więc `urllib` bez przeglądarki daje zero rekordów i do zebrania danych potrzebny jest Selenium (albo bezpośrednie wczytanie plików JSON, które nie zostało zaimplementowane).

6.2 Okno vs headless

Źródło: linie `PODSUMOWANIE:` i `PRZEBIEG:` w logach `…_20261018_165547.log` (okno) i `…_20261018_171656.log` (headless), przebiegi powtórzone 2026-10-18.

Miara	okno	headless	Stosunek (headless/okno)
Czas całkowity (s)	362,46 (06:02)	347,47 (05:47)	0,96
Rekordów w eksporcie	420	420	1,00
Błędy / retry	0 / 0	0 / 0	
s na rekord	0,86	0,83	0,96
Pozycje na liście / po dedup.	432 / 420	432 / 420	

Oba przebiegi dały ten sam wynik (420 rekordów, 0 błędów, 0 retry; SHA-256 plików CSV, JSON i JSONL identyczne), więc tryb nie wpływa na poprawność. Headless był szybszy o 14,99 s (ok. 4%). **Czas jest zdominowany przez pauzy grzecznościowe 0,5 s i losowe opóźnienie porcji serwisu (0,3-2 s), więc taka różnica nie pozwala wnioskować o przewadze żadnego trybu.** Potwierdza to wcześniejsza para przebiegów z 2026-10-13 (logi `…_20261013_001635.log` i `…_20261013_002240.log`): okno 351,59 s, headless 410,30 s, czyli kierunek różnicy był wtedy odwrotny (headless wolniejszy o 58,71 s, ok. 17%). Dwie pary dają różnice o przeciwnych znakach, więc sprawozdanie nie podaje średniej ani rozrzutu i nie twierdzi o przewadze żadnego trybu; to za mało powtórzeń na wniosek statystyczny.